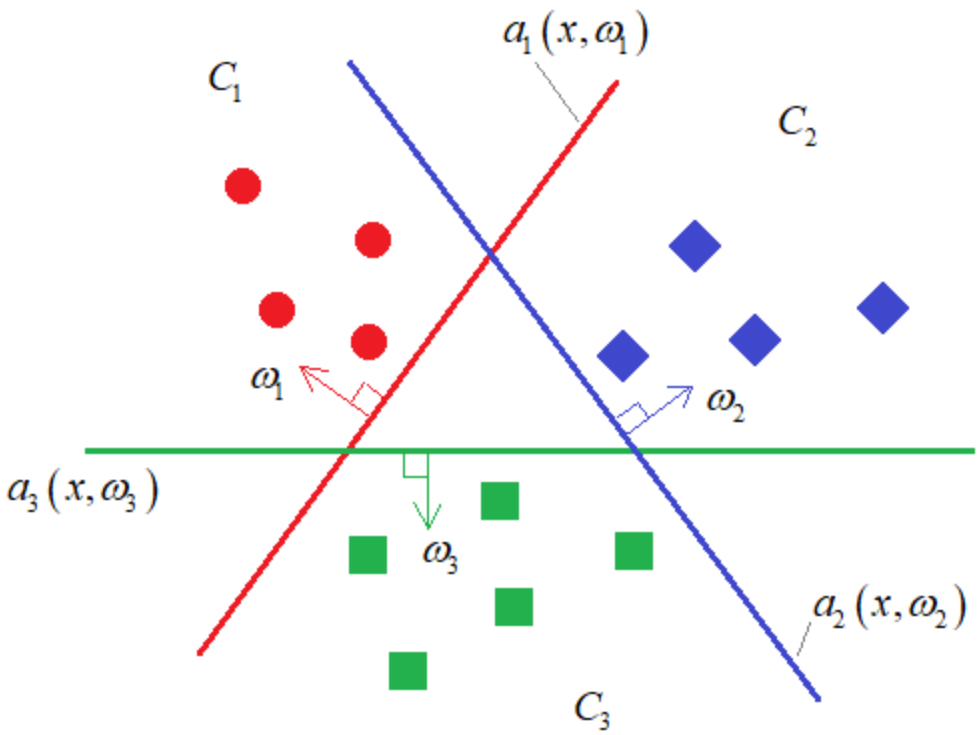
Многоклассовая классификация (добавление к линейной классификации)

Пусть дана некоторая выборка и ответы на ней. Ответы находятся в множестве от 1 до K (классов).
Нужно построить алгоритм, который будет предсказывать, к какому из этих К классов относится каждый объект.
Два вопроса:
1) как построить такой алгоритм
2) как измерить его качество.
Существуют подходы для построения многоклассового алгоритма классификации «напрямую». Однако эти подходы довольно сложные.
Попробуем воспользоваться имеющимся инструментарием, то есть свести задачу многоклассовой классификации к задаче бинарной классификации.

Метод "один против всех"



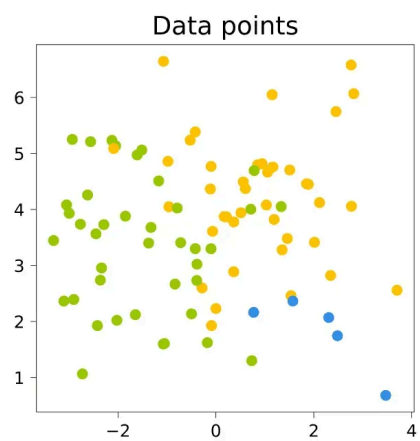
K классов $Y = \{1, \ldots, K\}$
$X_k = (x_i, [y_i = k])_{i=1}^{l}$
Обучаем К линейных классификаторов, выдающих оценки принадлежности классам $1, \ldots, k$ соответственно. Классификатор с номером k будет отличать класс k от всех остальных.
a_k – оценка принадлежности классу k, например, $< w_k, x >$

Итоговый классификатор должен возвращать класс, соответствующий самому уверенному из бинарных алгоритмов

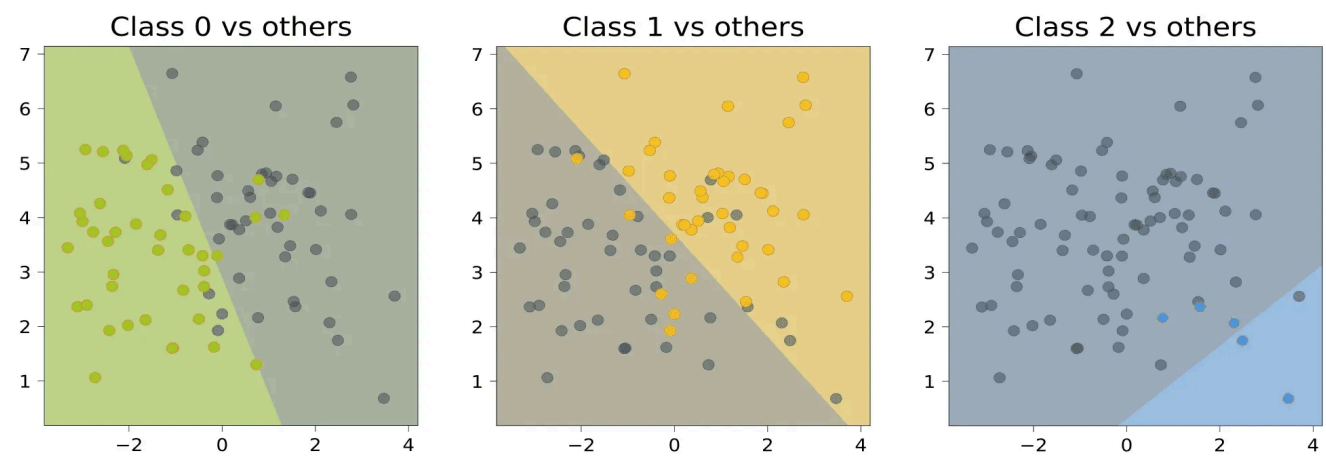
$$a(x) = \arg\max_{k=1,\ldots,K} a_k(x)$$

Идея состоит в том, чтобы построить к классификаторов, каждый из которых будет отделять один класс от всех остальных.
Возьмём класс k. Для обучения используем всю выборку. Целевая переменная индикатор того, что объект x относится к классу k. Обучим модель a_k - она будет отделять k класс от всех остальных.
Имеем к моделей. Как их объединить?
Найдем для объекта x такую k, для которого a_k выдала максимальную оценку.

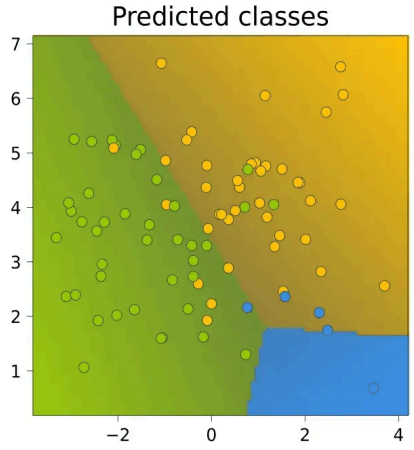
Пример



Обучение трех классификаторов



Итог



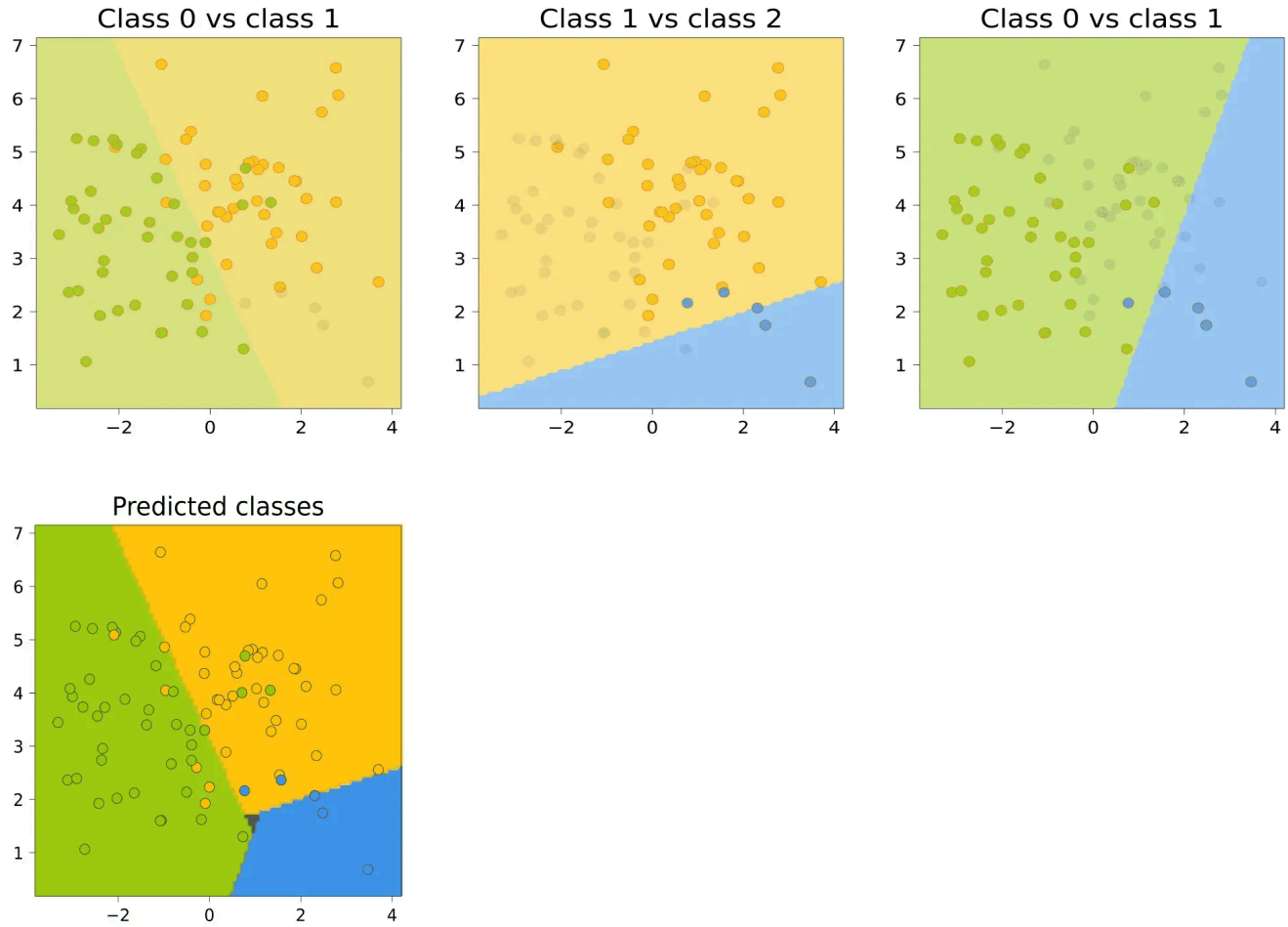
Видно, что самый маленький класс «обидели». Проблема данного подхода заключается в том, что каждый из классификаторов $b_1(x), \ldots, b_k(x)$ обучается на своей выборке, и значения линейных функций $< w_k, x > + w_{0k}$ или, проще говоря, "выходы" классификаторов могут иметь разные масштабы. Из-за этого сравнивать их будет неправильно. Нормировать вектора весов, чтобы они выдавали ответы в одной и той же шкале, не всегда может быть разумным решением.

Метод "все против всех"

Идея: устроить турнир между классами.
Возьмем все пары классов и построим для каждой пары из класса k и класса такую обучающую выборку, что в ней будут только объекты, которые относятся либо к классу k, либо к классу .
Для каждой пары k построим свой классификатор a_k. Он может выдавать просто класс, не обязательно оценку принадлежности.
Как определить, к какому классу отнести объект?
Берем какой-нибудь класс k и смотрим, в паре со сколькими классами он победил. То есть мы берем все пары, где есть класс k, проводим там матчи, смотрим, куда классификатор отнесет наш объект, считаем, сколько было побед и относим к тому классу k, который достиг наибольшего количества побед.
$x_k = [(x_i, y_i) \in X y_i = k или y_i =]$
Обучаем $a_k(x)$ на X_k
Итоговая модель:

$$a(x) = \arg \max_{k \in (1, \ldots, K)} \quad {}_{K}{}_{=1} [a_k(x) = k]$$

Пример



Обратите внимание на серый треугольник на стыке областей. Это точки, для которых голоса разделились (в данном случае каждый классификатор выдал какой-то свой класс, то есть у каждого класса было по одному голосу). Для этих точек нет явного способа выдать обоснованное предсказание.
Проблема: нужно обучить порядка K^2 классификаторов. Но при этом каждый из них обучается на меньшей выборке.

Как считать метрики качества?

Самая простая метрика качества – это доля правильных ответов или доля ошибок, если мы хотим минимизировать. Эта формула подходит для любого количества классов.
Доля неправильных ответов

$$1l \quad l_{i=1}[a(x_i) \quad y_i]$$

Доля правильных ответов:

$$auay(a, X) = 1l \quad l_{i=1}[a(x_i) = y_i]$$

Другие метрики сложнее: точность, полнота, F-мера,
площади под кривыми и их как-то надо обобщить.

Есть два способа: микроусреднение и макроусреднение.
Пусть выборка состоит из K классов. Рассмотрим K двухклассовых задач, каждая из которых заключается в отделении своего класса от остальных, то есть целевые значения для k-й задачи вычисляются как $y_i^k = [y_i = k]$. Для каждой из них различные характеристики (TP, FP, и т.д.) алгоритма $a^k(x) = [a(x) = k]$; будем обозначать эти величины как $T_k, {}_k, {}_k, T_k$.
Заметим, что в двухклассовом случае все метрики качества, которые мы изучали, выражались через эти элементы матрицы ошибок.
Микроусреднение:

1. Вычисляем $T_k, {}_k, {}_k, T_k$
2. Суммируем по всем классам, получаем TP, FP, и т.д.
3. Подставляем в формулу для precision, recall и т.д.

Замечание. Крупные классы вносят больший вклад.
$T = {}_{1K} {}_{k=1}^{K} T_k$
$ii(a, X) = \ {}_T \ {}_{T+}$
Макроусреднение:

1. Вычисляем нужную метрику для каждого класса ii_k, precision и т.д
2. Суммируем и усредняем: по всем классам

Замечание. Игнорирует размеры классов.
$ii_k(a, X) = {}_{T_k T_k +_k}$
$ii(a, X) = {}_{1K} {}_{k=1}^{K} ii_k(a, X)$

Замечание

Если какой-то класс имеет очень маленькую мощность, то при микро-усреднении он практически никак не будет влиять на результат, потому что его вклад в средние TP, FP, FN и TN будет незначителен. В случае же с макро-вариантом усреднение проводится для величин, которые не чувствительны для соотношения классов (точноть и полнота, например) и поэтому каждый класс внесет равный вклад в итоговую метрику.

Ансамбли деревьев

Решающее дерево

- Сильно переобучается
- Сильно меняется при небольшом изменении выборки

Идея ансамбля

Теорема Кондорсе «о жюри присяжных» (1784). Если каждый член жюри присяжных имеет независимое мнение, и если вероятность правильного решения члена жюри больше 0.5, то тогда вероятность правильного решения присяжных в целом возрастает с увеличением количества членов жюри и стремится к единице. Если же вероятность быть правым у каждого из членов жюри меньше 0.5, то вероятность принятия правильного решения присяжными в целом монотонно уменьшается и стремится к нулю с увеличением количества присяжных.
Фрэнсис Гальтон в 1906 году посетил рынок, где проводилась некая лотерея для крестьян. Их собралось около 800 человек, и они пытались угадать вес быка, который стоял перед ними. Бык весил 1198 фунтов. Ни один крестьянин не угадал точный вес быка, но если посчитать среднее от их предсказаний, то получим 1197 фунтов (**мудрость толпы**).

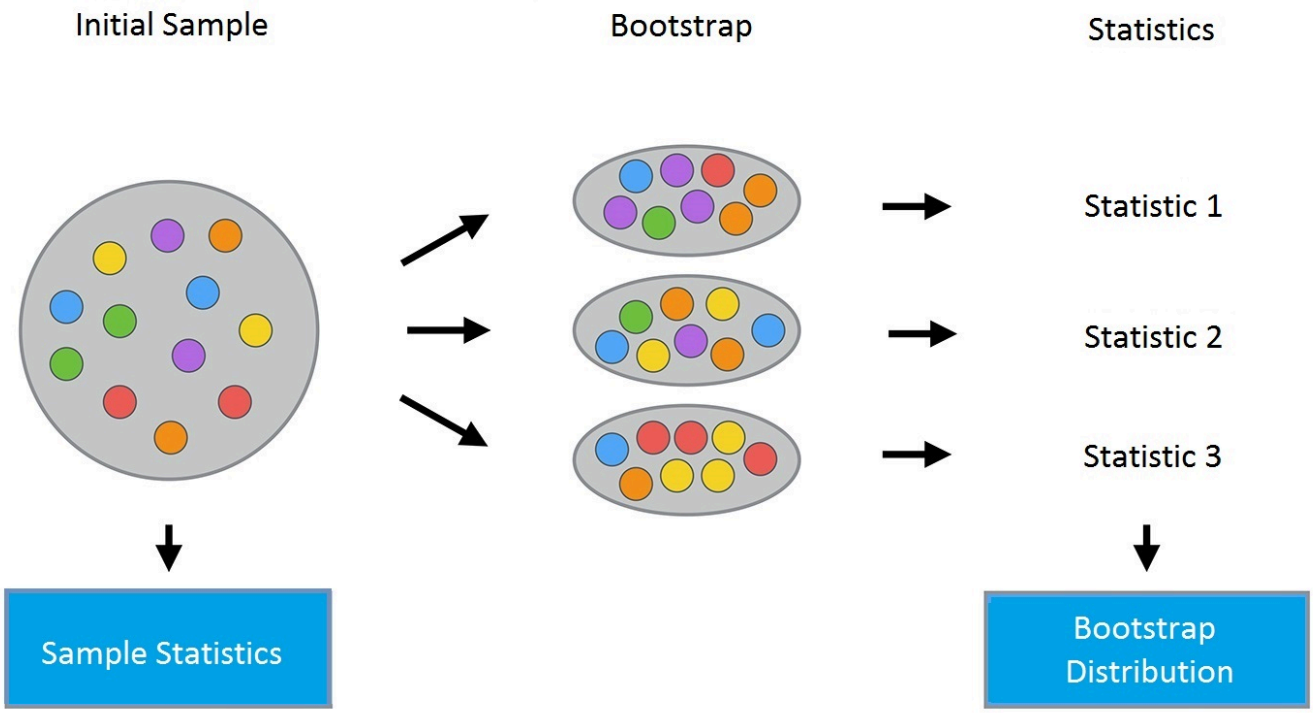
Композиция деревьев

1 вариант

Композиция – это объединение алгоритмов в один.
Обучим много деревьев: $b_1(x), \ldots, b(x)$
Усредним ответы: $a(x) = {}_1{}_{=1} b(x)$
Для классификации добавим знак: $a(x) = i_1{}_{=1} b(x)$

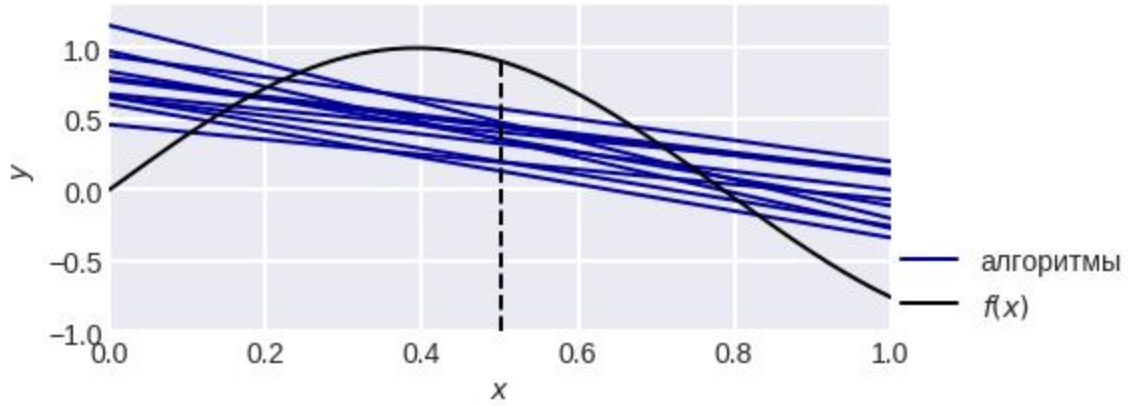
Прогнозы деревьев: -1, -1, 1, -1, 1, -1
$a(x) = i - {}_2 = -1$
Ответ: -1, так как за него "проголосовало" большинство деревьев.
N базовых алгоритмов **нельзя обучать на всей обучающей выборке**, так как в этом случае они получаются одинаковыми, и в их усреднении не будет никакого смысла.
Один из способов сделать базовые алгоритмы различными – использовать **рандомизацию**, то есть **обучать базовые алгоритмы на разных подвыборках** обучающей выборки. А поскольку решающие деревья сильно меняются даже от небольших изменений обучающей выборки, такая рандомизация значительно повышает различность базовых алгоритмов.

Бутстрап (bootstrap) – один из популярных подходов к построению подвыборок. Он заключается в том, что из обучающей выборки длины l выбирают с возвращением l объектов. При этом новая выборка также будет иметь размер l, но некоторые объекты в ней будут повторятся, а некоторые объекты из исходной выборки в нее не попадут. Можно показать, что в бутстрапированной выборке будет содержаться в среднем 63% различных объектов исходной выборки.

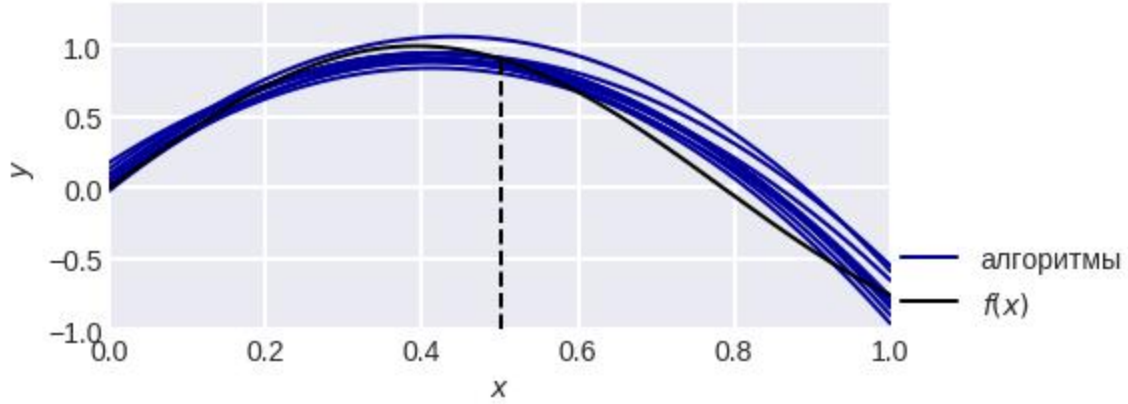


Другой подход к рандомизации – **генерация случайного подмножества обучающей выборки**. **Размер** этого случайного подмножества является **гиперпараметром**.
Например, можно случайно взять половину исходной выборки и обучить на ней базовый алгоритм.
Этот подход несколько проигрывает бутстрапу, так как содержит гиперпараметр, в то время как бутстрап без какой-либо настройки выдает подвыборку.
Ошибка алгоритма на новых тестовых данных складывается из **трех компонент: шума, смещения и разброса**. При этом все они характеризуют разные аспекты данных и модели, с помощью которой решается задача на этих данных
Шум — является характеристикой данных и будет проявляться, какая бы модель не использовалась.
Смещение и разброс связаны с используемой моделью. Пусть обучающая выборка генерируется из некоторого вероятностного распределения. На каждой конкретной обучающей выборке можно обучить некоторую модель и использовать обученную модель на тестовой выборке. Тогда:
Смещение (bias) – отклонение усредненного по различным обучающим выборкам, прогноза заданной модели от прогноза идеальной модели.
Разброс (variance) – дисперсия ответов моделей, обученных по различным обучающим выборкам. Разброс характеризует то, насколько сильно прогноз алгоритма зависит от конкретной обучающей выборки
Смещение показывает, насколько хорошо с помощью данного алгоритма можно приблизить истинную зависимость, а разброс характеризует чувствительность алгоритма к изменениям в обучающей выборке.
На первом рисунке показаны различные полиномы первой степени, они настроены на разных обучающих выборках. В точке $x = 05$ ответы алгоритмов являются случайными

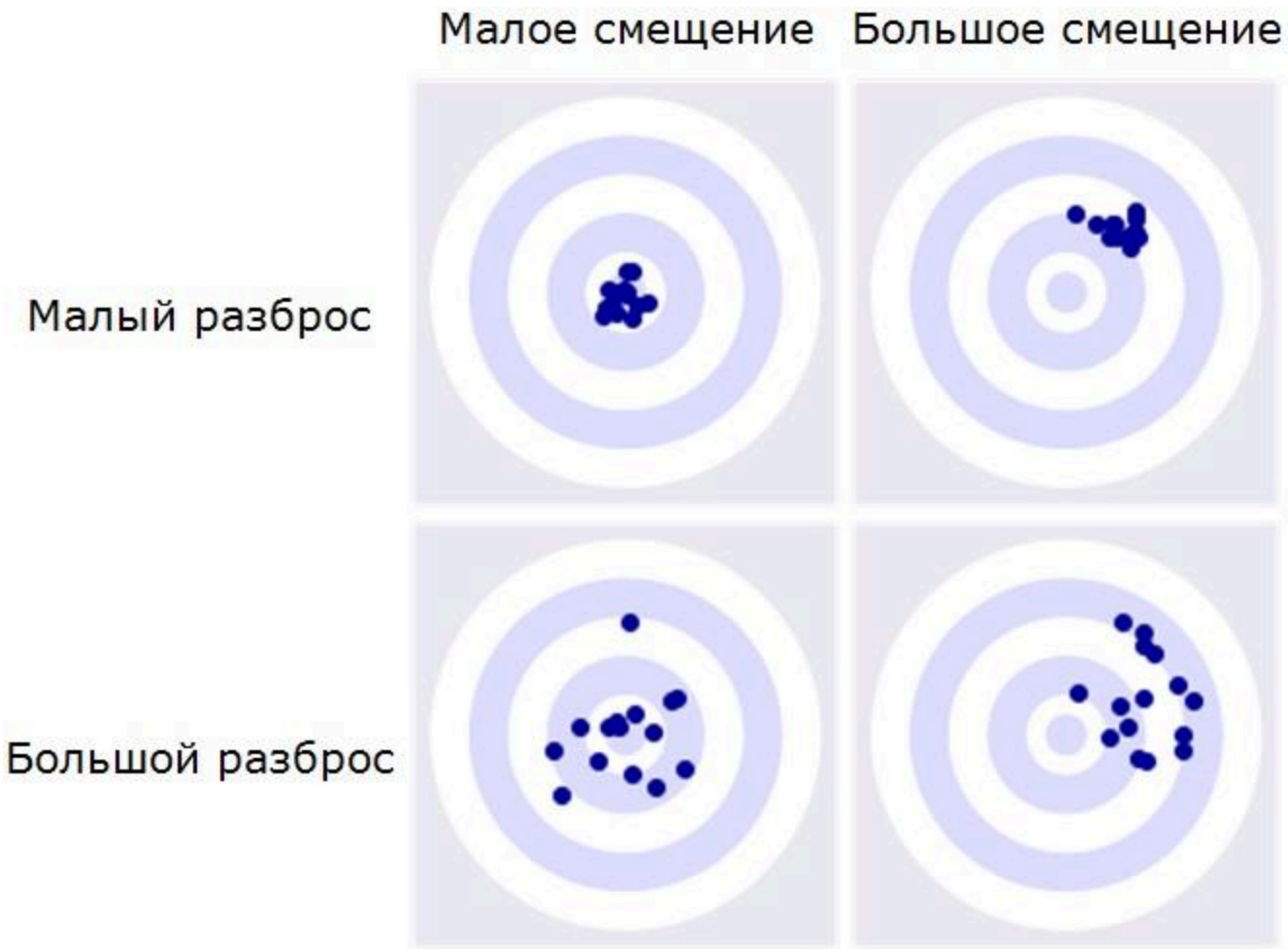
величинами, они немного «разбросаны» (есть variance), а также они сильно смещены (есть bias) относительно правильного ответа.



На втором рисунке изображены уже полиномы второй степени (настроенные на тех же выборках). В точке $x = 05$ у них сильно меньше смещение и чуть меньше разброс. Видно, что они совсем неплохо описывают целевую зависимость во всех точках.



Пример из жизни: игра в дартс



Однако это аналогия не совсем точная, например:
Известно, что спортсменов учат стрелять кучно, поскольку нужного смещения легко добиться (например, целясь повыше). С алгоритмами машинного обучения всё сложнее. Смещение нельзя просто «подвинуть», приходится переходить к другой (например, более сложной модели), а у неё может быть уже другой разброс.
Спортсмен может повысить точность целясь выше/ниже/правее/левее, а для алгоритма нет таких понятий. Напомним, что разброс и смещение мы вводили в конкретной точке. Если изменить смещение в этой точке, то модель будет по-новому вести себя и в остальных. Если же усреднить смещение, точнее его квадрат, по всем точкам, то мы получим просто число (оно не указывает, как менять модель, чтобы уменьшить ошибку).

Линейные модели **способны** восстанавливать только линейные зависимости, а, следовательно, в случае нелинейных задач, которых подавляющее большинство смещение при использовании таких алгоритмов будет большим. Разброс, наоборот, будет маленьким из-за малого числа параметров, сравнимого с количеством признаков. Вряд ли параметры линейной модели сильно поменяются при незначительном изменении обучающей выборки.
Для метода **k ближайших соседей** при увеличении к **смещение увеличивается, дисперсия уменьшается** (чем больше соседей, тем более изменчивым становится его предсказание, то есть в каком-то смысле сложность убывает)

Решающие деревья – характеризуются **низким смещением**, то есть способны восстанавливать сложные закономерности, и **большим разбросом**: решающие деревья сильно меняются даже при небольших изменениях обучающей выборки (это верно начиная с определенной глубины).
Утверждение **bias-variance trade-off**: чем выше сложность обучаемой модели, тем меньше её смещение и тем больше разброс (не является истиной для всех моделей, например, для нейронных сетей с ростом сложности может уменьшаться и то, и другое).
Например, **деревья маленькой глубины** могут иметь **высокое смещение** и **низкий разброс**, а **глубокие** – **наоборот**.

Смещение и разброс композиции

Смещение композиции совпадает со смещением отдельного базового алгоритма.
Таким образом, поскольку глубокие деревья характеризуются низким смещением, то же самое будет верно и для композиции глубоких деревьев.
Разброс композиции уже отличается от разброса одного базового алгоритма:
Разброс $= 1$ (разброс базового алгоритма) + (корреляция между базовыми алгоритмами)
Если алгоритмы независимы, то есть их прогнозы не коррелируются между собой, то разброс уменьшается в раз.

Уменьшение корреляции базовых алгоритмов

Обучение базовых алгоритмов происходит на случайных подвыборках обучающей выборки. Причем чем меньше размер случайной подвыборки, тем более независимыми получаются базовые алгоритмы (бэггинг).
Выбирается случайное подмножество признаков (столбцов матрицы «объекты -признаки») и очередной базовый алгоритм обучается только на этих признаках. Доля выбираемых признаков является гиперпараметром этого метода (метод случайных подпространств).
Оба этих подхода можно объединить и использовать одновременно.

Бэггинг

Пусть имеется обучающая выборка X. С помощью бутстрапа сгенерируем из неё выборки $X_1, \ldots, X$. Теперь на каждой выборке обучим свой классификатор $a(x)$. Итоговый классификатор будет усреднять ответы всех этих алгоритмов (в случае классификации это соответствует голосованию)

$$a(x) = 1 \quad {}_{i=1} a_i(x)$$

Бэггинг позволяет снизить дисперсию (variance) обучаемого классификатора, уменьшая величину, на сколько ошибка будет отличаться, если обучать модель на разных наборах

данных, Эффективность бэггинга достигается благодаря тому, что базовые алгоритмы, обученные по различным подвыборкам, получаются достаточно различными, и их ошибки взаимно компенсируются при голосовании, а также за счёт того, что объекты-выбросы могут не попадать в некоторые обучающие подвыборки.
Бэггинг эффективен на малых выборках, когда исключение даже малой части обучающих объектов приводит к построению существенно различных базовых классификаторов. В случае больших выборок обычно генерируют подвыборки существенно меньшей длины.

Случайный лес

Метод случайных лесов основан на бэггинге над решающими деревьями. Бэггинг сильнее уменьшает дисперсию базовых алгоритмов, если они слабо коррелированы. В случайных лесах корреляция между деревьями понижается путем рандомизации по двум направлениям: по объектам и по признакам.
Во-первых, **каждое дерево обучается по бутстрапированной подвыборке**.
Во-вторых, **в каждой вершине разбиение ищется по подмножеству признаков**.
Вспомним, что при построении дерева последовательно происходит разделение вершин до тех пор, пока не будет достигнуто идеальное качество на обучении. Каждая вершина разбивает выборку по одному из признаков относительно некоторого порога. **В случайных лесах признак, по которому производится разбиение, выбирается не из всех возможных признаков, а лишь из их случайного подмножества размера m**.

Алгоритм построения случайного леса

1: для $= 1, \ldots,$
2: Сгенерировать выборку X с помощью бутстрэпа
3: Построить решающее дерево $b(x)$ по выборке X:

- дерево строится, пока в каждом листе не окажется не более $_i$ объектов
- при каждом разбиении сначала выбирается случайных признаков из , и оптимальное разделение ищется только среди них

4: Вернуть композицию $a(x) = {}_1{}_{=1} b(x)$

Для задачи классификации мы выбираем решение голосованием по большинству, а в задаче регрессии — средним.
Регрессия: $a(x) = {}_1{}_{=1} b(x)$
Классификация: $a(x) = \mathrm{g}_1{}_{=1} b(x)$

Параметры для случайного леса

В качестве базовых алгоритмов нужно использовать **глубокие деревья**.
Количество признаков. Чем больше признаков, тем больше корреляция между

деревьями. Чем меньше признаков, тем слабее сами деревья. На практике рекомендуется брать корень из числа всех признаков для классификации и треть признаков для регрессии.
Количество деревьев в ансамбле. Увеличение числа элементарных алгоритмов в ансамбле не меняет смещения и уменьшает разброс. Так как число признаков и варианты подвыборок, на которых строятся деревья в случайном лесе, ограничены, уменьшать разброс до бесконечности не получится. Поэтому имеет смысл построить график ошибки от числа деревьев и ограничить размер леса в тот момент, когда ошибка перестанет значимо уменьшаться. Кроме того, надо учитывать время работы.

Проблемы со случайными лесами

Обучение глубоких деревьев требует **очень много вычислительных ресурсо**в, особенно в случае большой выборки или большого числа признаков.
Если ограничить глубину решающих деревьев в случайном лесе, то они уже не смогут улавливать сложные закономерности в данных. Это приведет к тому, что сдвиг будет слишком большим.
Процесс построения деревьев является ненаправленным: каждое следующее дерево в композиции никак не зависит от предыдущих. Из-за этого для решения сложных задач необходимо огромное количество деревьев.

Бустинг (2 вариант построения ансамбля)

Базовые алгоритмы строятся последовательно, один за другим.
Каждый следующий алгоритм строится таким образом, чтобы исправлять ошибки уже построенной композиции.
Благодаря тому, что построение композиций в бустинге является направленным, достаточно использовать простые базовые алгоритмы, например, неглубокие деревья.

Пример

Пусть дана задача регрессии, в которой в качестве ошибки используется среднеквадратичная ошибка:
$E(a, X) = 1l \sum_{i=1}^{l}(a(x_i) - y_i)^2$
Для начала необходимо обучить первый простой алгоритм (например, неглубокое решающее дерево):

$$b_1(x) = \arg\min_b 1l \sum_{i=1}^{l}(b(x_i) - y_i)^2$$

Такая задача минимизации квадратичной ошибки легко решается, например, градиентным спуском. Этот алгоритм будет первым алгоритмом в строящейся композиции. Второй алгоритм должен быть обучен таким образом, чтобы композиция первого и второго алгоритмов:

$b_1(x_i) + b_2(x_i)$
имела наименьшую из возможных ошибку на обучающей выборке:

$$b_2(x) = \arg\mathrm{m}_b 1l \quad {}_{l\ i=1}(b_1(x_i) + b(x_i) - y_i)^2 = \arg\mathrm{m}_b 1l \quad {}_{l\ i=1}(b(x_i) - (y_i - b_1(x_i)))^2$$

Другими словами, алгоритм $b_2(x)$ улучшает качество работы алгоритма $b_1(x)$. Продолжая по аналогии на шаге очередной алгоритм $b(x)$ будет определяться следующим образом:

$$b(x) = \arg\mathrm{m}_b 1l \quad {}_{l\ i=1}(b(x_i) - (y_i - {}_{-1\ =1} b(x_i)))^2$$

Процесс продолжается до тех пор, пока ошибка композиции $b_1(x) + \ + b(x)$ не будет устраивать.
Итак, каждый следующий базовый алгорит в бустинге обучается так, чтобы уменьшить общую ошибку всех своих предшественников.
Поэтому итоговая композиция будет иметь меньшее смещение, чем каждый отдельный базовый алгоритм (уменьшение разброса также может происходить).
Следовательно, в качестве базовых алгоритмов часто выбирают алгоритмы с высоким смещением и небольшим разбросом.
Например, если в качестве базовых классификаторов выступают деревья, то их глубина должна быть небольшой – обычно не больше 2-3 уровней.
Кроме того, модели с высоким смещением, как правило, быстрее учатся. Это важно для их последовательного обучения, которое может стать очень дорогим по времени, если на каждой итерации будет учиться сложная модель.

Стекинг (3 вариант)

Может использовать алгоритмы разного типа, а не только из какого-то фиксированного семейства.Например, в качестве базовых алгоритмов могут выступать метод ближайших соседей и линейная регрессия.
Результаты базовых алгоритмов объединяются в один с помощью обучаемой мета-модели, а не с помощью какого-либо обычного способа агрегации (суммирования или усреднения)
С точки зрения смещения и разброса стекинг не имеет прямой интерпретации, так как не минимизирует напрямую ни ту, ни другую компоненту ошибки. Удачно работающий стекинг просто уменьшает ошибку, и, как следствие, её компоненты тоже будут убывать.

Важность признаков (Feature Importance) и интерпертируемость моделей

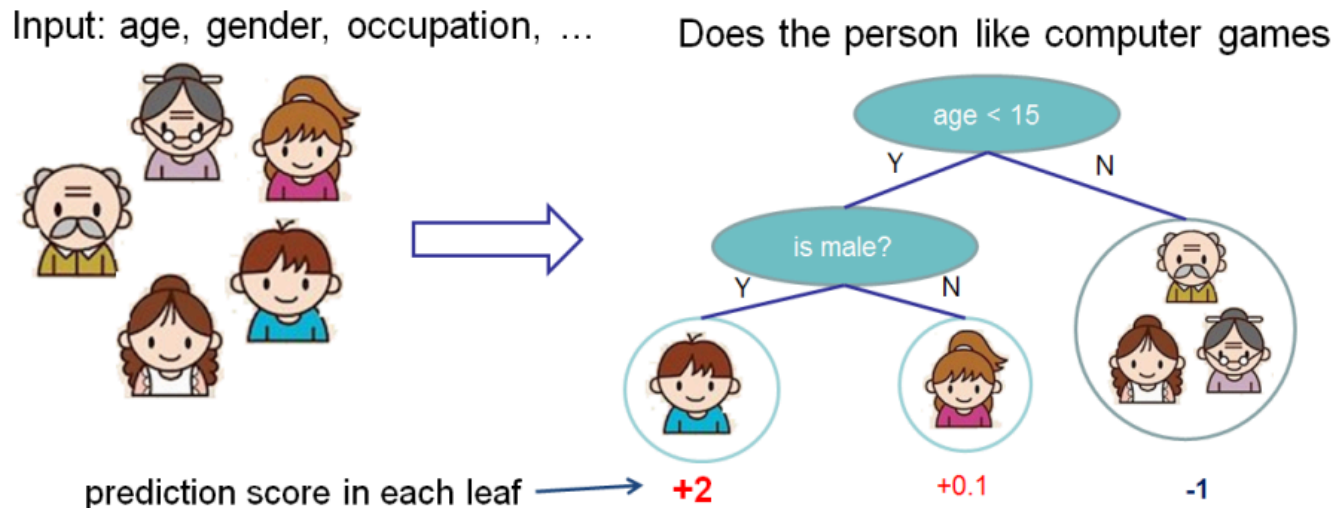
Важность признаков – это оценка вклада признака в предсказательную силу модели. Какое влияние оказывает признак на прогноз модели?

Зачем нужно оценивать важность признаков

Интерпретируемость: почему модель принимает те или иные решения (особенно важно в медицине, банках).
Анализ данных: понять природу изучаемого явления (например, что цена на квартиру сильнее всего зависит от расположения, а не от высоты потолков).
Улучшение модели (отбор признаков): отбросить бесполезные признаки, чтобы ускорить обучение и избежать переобучения.

Интерпертируемость модели

Прогностическая модель называется **интерпретируемой** (interpretable) или **объяснимой** (explainable), если человек может проинтерпретировать её решения. Почему алгоритм принял такое решение? Чем обусловлен такой прогноз?
Как правило, более сложные модели, учитывающие большее число факторов нелинейным образом, являются менее интерпретируемыми, но обеспечивают более высокую точность, поэтому существует противоречие между объяснимостью и качеством прогнозов.
Пример. Нравится ли человеку компьютерные игры.



Признаки – пол, возраст, работа.
Используется XGBoost.
Глобальный уровень. Анализируется набор данных в целом. Используется встроенная функция, которая подсчитывает, сколько раз признак использовался для разбиения
Локальный уровень. В общем случае скорее всего возраст окажется самым важным признаком. Однако если Игорю 50 лет и он работает тестировщиком игр, вероятно его работа будет играть большую роль.

Предположим, что теперь мы работаем с моделью линейной регрессии
В данном случае если весовой коэффициент при признаке имеет большое абсолютное значение, то признак оказал большое влияние на конечный результат.
Однако данная модель может выявлять только линейные зависимости. Например, вполне возможно, что возраст - важный признак, и если вам от 12 до 18 лет, то вы в гораздо

большей степени любите компьютерные игры, чем в любом другом возрасте (более младшем или более старшем возрасте), т.е. мы видим нелинейную взаимосвязь между возрастом и любви к компьютерным играм, которую линейная регрессия не сможет обнаружить.
Для поиска более сложных взаимосвязей понадобится более сложная модель. Однако **при использовании более сложных моделей, теряется легкость интерпретации**. Как только мы пытаемся обнаружить нелинейные взаимосвязи или даже взаимодействия (например, а что если возраст важен только в сочетании с полом?), сразу становится очень сложно интерпретировать модель.
Решение, лежащее между легко интерпретируемой моделью, позволяющей выявлять лишь простые взаимосвязи и сложной моделью, позволяющей находить очень интересные паттерны и при этом трудной для интерпретации – компромисс между интерпретируемостью сложностью.

Дополнительные материалы

1. Бутстрап, вставка и построение ансамблей – https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=5765871610321f83521c283980e47755fe793d1c, https://link.springer.com/article/10.1007/BFO0058655, https://link.springer.com/article/10.1023/A:1007563306331, https://habr.com/ru/companies/X5Tech/articles/679842/
2. Бэггинг и случайный лес – https://habr.com/ru/articles/801161/
3. Вывод формулы для бэггинга
4. Градиентный бустинг и его модификации – https://habr.com/ru/articles/799725L

Д/з

1. Имеются объекты трёх цветов: желтого – 21, зелёного – 20, синего – 4. Выполнялась классификация "один против всех". Модель определила "желтыми" 25 объектов, 20 из которых были действительно жёлтыми. "Синим" оказался только один объект, который на самом деле жёлтый. "Зелёными" – 19 объектов, все на самом деле зелёные. Подсчитать precision с помощью микроусреднения и макроусреднения. Сделать выводы.
2. По ссылке (начиная с подраздела "Реализации") разобрать и кратко написать о специфике видов бустинга.
3. Разобрать ансамблевые методы и их параметры в библиотеке sklearn.